

Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot

Abstract

MOONCAKE 是 Moonshot AI 开发的大语言模型聊天机器人服务 Kimi 的部署平台。该平台采用以 KVCache 为核心的解耦架构，不仅分离了预填充（prefill）集群与解码（decoding）集群，还高效利用 GPU 集群中未被充分开发的 CPU、DRAM、SSD 和网卡（NIC）资源，构建了分布式 KVCache。MOONCAKE 的核心是其以 KVCache 为中心的全局缓存，以及一款在严格遵守延迟相关服务等级目标（SLO）的前提下最大化吞吐量的调度器。

实验结果表明，MOONCAKE 在长上下文输入场景中表现卓越。基于真实负载的测试显示，与基准方法相比，MOONCAKE 在满足 SLO 要求的同时，将有效请求处理能力提升了 59%~498%。目前，MOONCAKE 已在数千个节点上投入运行，每日处理超过 1000 亿个 token。在实际部署中，MOONCAKE 的创新架构使 Kimi 在 NVIDIA A800 和 H800 集群上，分别比原有系统多处理 115% 和 107% 的请求。

1 Introduction

随着大型语言模型（LLM）在各类场景中的快速应用 [1-4]，LLM 部署的工作负载已呈现出显著的多样化特征。这些工作负载在输入 / 输出长度、到达分布上存在差异，更重要的是，它们对服务等级目标（SLO）的需求各不相同。作为模型即服务（MaaS）提供商，Moonshot AI 开发的 Kimi [5] 的核心目标之一，是在多重复杂约束下解决优化问题：优化目标是最大化整体有效吞吐量（这直接影响收益），而约束条件则体现为不同层级的 SLO——这些 SLO 通常涉及延迟相关要求，主要包括首 token 生成时间（TTFT）和 token 间生成时间（TBT）。

要实现这一目标，前提是充分利用 GPU 集群中可用的各类资源。具体而言，尽管当前 GPU 服务器多以高度集成的节点形式提供（例如 DGX/HGX 超级计算机 [6]），但仍需将其解耦并重组为多个分布式资源池，每个资源池针对不同但协同的目标进行优化。例如，已有众多研究者 [7-9] 提出将预填充（prefill）服务器与解码（decoding）服务器分离，因为 LLM 部署的这两个阶段具有截然不同的计算特性。

在这一解耦策略的基础上，我们进一步构建了分布式 KVCache（即 MOONCAKE Store）：通过整合 GPU 集群的 CPU、DRAM、SSD 和 RDMA 资源实现。这种创新架构利用未充分利用的资源，构建了高效的近 GPU 前缀缓存，显著提升了全局缓存容量和节点间传输带宽。最终形成的分布式 KVCache 系统，贯彻了“以更多存储换取更少计算”的核心原则。如图 1 所示，在众多重要的实际场景中，该系统能在满足所需 SLO 的前提下，大幅提升 Kimi 的最大吞吐量。本文后续将首先通过数学分析，阐述该策略对 LLM 部署的收益，并结合真实数据进行实证评估（详见 2.2 节）；随后详细介绍 PB 级分布式缓存的实现方案——该缓存通过速率高达 8×400 Gbps 的 RDMA 网络实现互联（详见 3.2 节）。

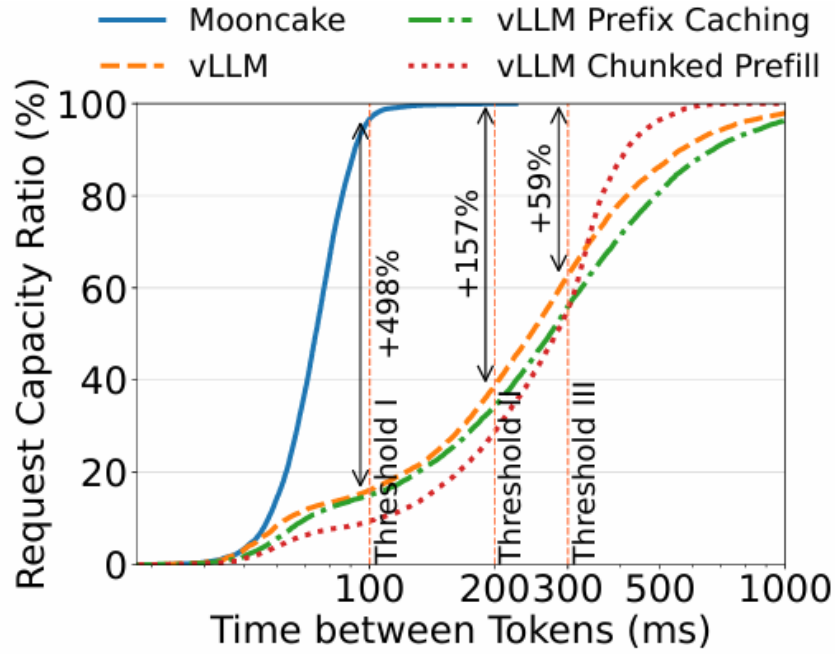


Figure 1: The experiment of the effective request capacity of MOONCAKE under the real-world conversation workload and different TBT SLOs. In this experiment, MOONCAKE and three baseline systems utilize 16 $8\times$ A800 nodes each. More on §5.2.

基于这一思路，我们还发现 KVCache 的调度是 LLM 部署的核心环节，因此提出了相应的解耦架构。图 2 展示了我们当前用于 LLM 部署的、以 KVCache 为核心的解耦架构，将其命名为 MOONCAKE。对于每个请求，全局调度器（Conductor）会选择一对预填充实例与解码实例，并按以下步骤调度请求：1) 将尽可能多的可复用 KVCache 传输至选定的预填充实例；2) 以分块 / 分层方式完成预填充阶段，并将生成的 KVCache 持续流式传输至对应的解码实例；3) 在解码实例中加载 KVCache，并将请求加入持续批处理流程以生成输出结果。

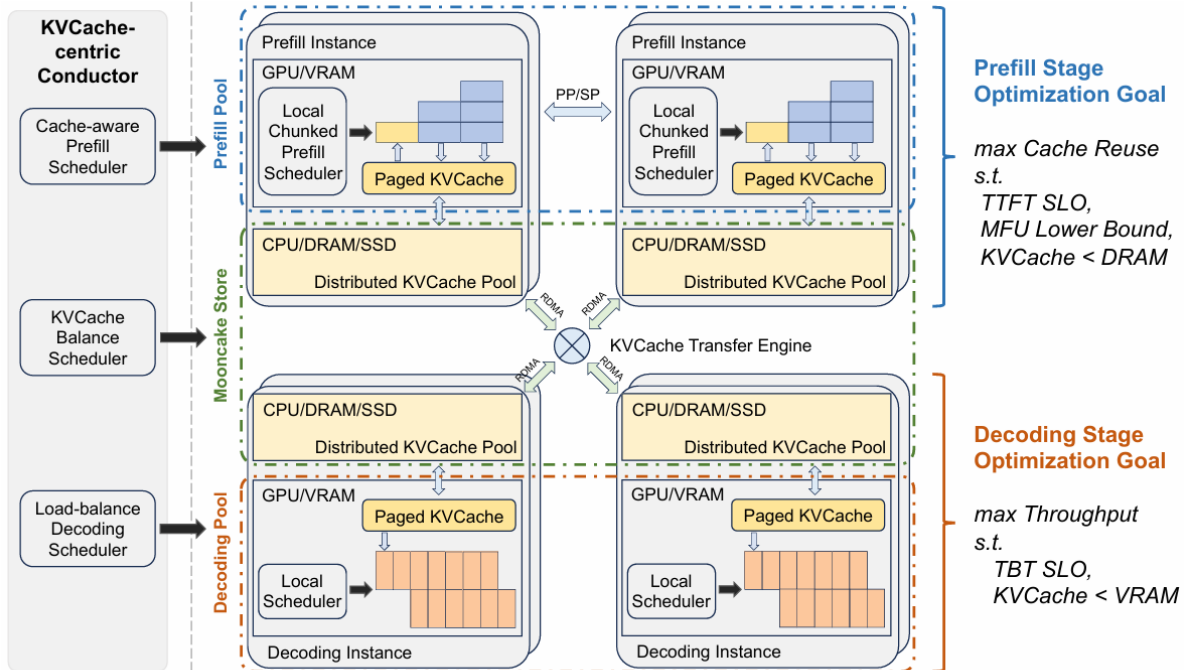


Figure 2: MOONCAKE Architecture.

尽管上述流程看似直观，但其选择策略因多重限制而较为复杂。在预填充阶段，核心目标是尽可能复用 KVCache 以避免冗余计算，但分布式 KVCache 池在容量和访问延迟方面均面临挑战。因此，Conductor 需实现“感知 KVCache”的请求调度，并执行缓存交换、复制等操作：访问热度最高的缓存块需复制到多个节点以避免获取拥堵，而热度最低的缓存块则需交换出去以降低存储成本。与之相反，解码阶段有着不同的优化目标与约束条件：其目标是在解码批处理中聚合尽可能多的 token，以提升模型 FLOPs 利用率（MFU），但这一目标不仅受 TBT SLO 限制，还受 VRAM 可容纳的聚合 KVCache 总大小约束。

在第 4 节中，我们将详细介绍以 KVCache 为核心的请求调度算法——该算法在实例负载与用户体验（以 TTFT 和 TBT SLO 衡量）之间实现平衡，其中包括基于启发式的自动热点迁移方案：无需精确预测 KVCache 的未来使用情况，即可实现热点 KVCache 块的复制。实验结果表明，在实际场景中，这种以 KVCache 为核心的调度策略能显著降低 TTFT。

我们还将介绍实现过程中的主要设计选择，尤其是当前研究中未涉及的部分。例如，在预填充 / 解码（P/D）解耦方面，由于带宽需求及分块预填充（chunked prefill）相关的权衡问题（如 Sarathi-Serve [10] 所提及），目前学界对其大规模实践可行性仍存在争议。通过与 vLLM 的对比实验我们证明：借助高度优化的传输引擎，可有效应对通信挑战，且在 SLO 限制严格的场景中，P/D 解耦架构更具优势（详见 5.2 节）。此外，我们还将阐述如何实现独立的预填充节点池，以灵活应对上下文长度的动态分布：通过分块流水线并行（CPP）机制，将单个请求的处理任务分配到多个节点上执行——这对降低长上下文输入的 TTFT 至关重要。与传统的序列并行（SP）方案相比，CPP 不仅减少了网络开销，还降低了对频繁弹性扩容的依赖（详见 3.3 节）。

目前，MOONCAKE 已作为 Kimi 的部署平台投入使用，并成功应对了指数级增长的工作负载（每日处理 token 量超 1000 亿）。历史数据统计显示，MOONCAKE 的创新架构使 Kimi 在 A800 和 H800 集群上，分别能比原有系统多处理 115% 和 107% 的请求。

为确保实验结果的可复现性，同时保护专有信息，我们还提供了基于“模拟 LLaMA3-70B 架构的虚拟模型”的详细实验结果——该实验采用真实工作负载的重放轨迹。这些轨迹与 MOONCAKE 的 KVCache 传输基础设施已开源，可通过以下链接获取：<https://github.com/kvcache-ai/Mooncake>。

在基于公开数据集和真实工作负载的端到端实验中，MOONCAKE 在长上下文场景中表现突出：与基准方法相比，其在满足 SLO 的前提下，有效请求处理能力最高可提升 498%。在 5.3 节中，我们将 MOONCAKE Store 与本地缓存设计进行对比，发现 MOONCAKE Store 的全局缓存设计显著提升了缓存命中率——实验中，其缓存命中率最高可达本地缓存的 2.36 倍，进而使预填充计算时间减少了 48%。据我们所知，MOONCAKE 是首个在大规模部署场景中，证实“通过分布式 KVCache 池实现不同聊天会话与查询间 KVCache 共享”能带来显著收益的系统。此外，我们还对 MOONCAKE 中支持高速 RDMA 传输的传输引擎性能进行了评估，结果显示其速度分别约为现有方案的 2.4 倍和 4.6 倍（详见 5.4 节）。

2 预备知识与问题定义

2.1 大语言模型部署的服务等级目标

现代大型语言模型（LLM）基于 Transformer 架构构建，该架构通过注意力机制（attention mechanisms）和多层感知机（MLP）处理输入数据。目前主流的基于 Transformer 的模型（如 GPT [11]、LLaMA [12]）均采用“仅解码器”（decoder-only）结构。每个推理请求在逻辑上可划分为两个阶段：预填充阶段（prefill stage）与解码阶段（decoding stage）。

在预填充阶段，所有输入 token 会被并行处理，因此该阶段通常计算密集。预填充阶段会生成第一个输出 token，同时存储计算得到的键（key）和值（value）的中间结果——这一结果被称为 KVCache。随后的解码阶段会利用该 KVCache，以自回归（autoregressive）方式生成新 token。由于自回归生成的特性限制，解码阶段每批次仅能处理一个 token，这使得该阶段受内存约束显著，且计算时间随批次大小的增长呈亚线性（sublinearly）提升。因此，解码阶段普遍采用的优化方案是“持续批处理”（continuous

batching) [13,14]: 在每次迭代前, 调度器会检查请求状态, 将新到达的请求加入批处理队列, 同时移除已完成的请求。

由于预填充与解码阶段的特性差异显著, 模型即服务 (MaaS) 提供商针对两个阶段设定了不同的服务等级目标 (SLO) 衡量指标。具体而言, 预填充阶段的核心衡量指标是“请求到达至生成首个token的延迟”, 即首token生成时间 (TTFT, Time to First Token); 而解码阶段则聚焦于“同一请求的连续token生成之间的延迟”, 即token间生成时间 (TBT, Time Between Tokens)。

在实际部署中, 若监控系统检测到 SLO 未达标, 需通过两种方式调整: 要么增加推理资源, 要么拒绝部分新进入的请求。然而, 受当前 GPU 资源供应限制, 推理集群的弹性扩容通常难以实现。因此, 我们会主动拒绝那些预测无法满足 SLO 的请求, 以缓解集群负载压力。我们的核心目标是在遵守 SLO 约束的前提下最大化整体吞吐量——这一概念在其他研究中也被称为“有效吞吐量” (goodput) [8,15]。

2.2 以更多存储换取更少计算

为满足上述严苛的 SLO 要求, 一种广泛采用的解决方案是: 缓存此前生成的 KVCache, 当检测到前缀匹配 (prefix match) 时复用该缓存。然而, 现有方案 [16-18] 通常将缓存限制在本地高带宽内存

(HBM) 和 DRAM 中, 其核心假设是: 全局调度所需的传输带宽过高, 实际应用中难以实现。但正如我们将在 5.3 节中阐述的, 本地 DRAM 的容量仅能支持最高 50% 的理论缓存命中率, 因此构建全局缓存至关重要。本节将通过数学分析, 推导该策略所需的实际带宽, 解释为何分布式缓存 (尤其针对 LLaMA3-70B 这类大模型) 具备显著优势。更多实验结果将在 5.4.2 节中呈现。

我们的分析基于特定模型展开, 所使用的符号定义如表 1 所示, 同时融入了 LLaMA3-70B 模型的具体参数。当前主流 LLM 均为自回归语言模型, 其单个token的 KVCache 仅依赖自身及之前的token, 因此, 对应“相同输入前缀”的 KVCache 可直接复用, 且不会影响输出准确性。若当前请求的长度为 n 的提示词 (prompt), 与已缓存的 KVCache 存在长度为 p 的公共前缀, 则其预填充过程可按以下方式优化:

$$q[p : n], k[p : n], v[p : n] = MLP(hidden[p : n])$$

$$k[1 : n], v[1 : n] \leftarrow KVCache + (k[p : n], v[p : n])$$

$$o[p : n] = Attention(q[p : n], k[1 : n], v[1 : n])$$

$$KVCache \leftarrow (k[1 : n], v[1 : n])$$

表 1: 符号与参数定义。模型及硬件参数基于 LLaMA3-70B 模型与 8×A800 GPU 配置设定。

符号	描述	取值
l	层数 (Num layers)	80
d	模型维度 (Model dimension)	8192
a, b	公式 1 中的常数系数 (Constant coefficients in Equation 1)	4, 22
gqa	查询头数 / 键值头数 (Num q heads / Num kv heads)	8

符号	描述	取值
s	张量元素大小 (Tensor element size)	2 B (BFloat16)
G	GPU 计算吞吐量 (GPU computation throughput)	8 × 312 TFLOPS
B _{h2d}	主机到设备带宽 (Host-to-device bandwidth)	128 GB/s
B _{niC}	网卡带宽 (NIC bandwidth)	800 Gbps
n, p	提示词长度与匹配前缀长度 (Prompt and matched prefix length)	-

给定输入长度 n，预填充阶段的计算量 (FLOPS) 可按以下公式计算：

$$flops(n) = l \times (an^2d + bnd^2)$$

由此可知，复用 KVCache 可使预填充阶段的计算成本减少约 $l \times (ap^2d + bpd^2)$ ，但这需要将缓存的 KVCache 传输至预填充 GPU 的 HBM 中，传输数据量大小为 $p \times l \times (2 \times d / gqa) \times s$ 。假设平均计算吞吐量为 G，平均 KVCache 加载速度为 B (B 由 B_{h2d} 与 B_{niC} 中的较小值决定)，则当满足以下条件时，复用 KVCache 能降低 TTFT (具备实际收益)：

$$\frac{B}{G} > \frac{2ds}{gqa \times (apd + bd^2)}$$

在这类场景中，复用 KVCache 不仅能减少 GPU 计算时间与成本，还能通过缩短 TTFT 提升用户体验。值得注意的是，带宽相对于计算吞吐量的“收益阈值”更易满足——尤其当模型维度 d (与模型大小正相关) 更大时。例如，在搭载 8×A800 GPU 的机器上运行 LLaMA3-70B 模型，且前缀长度 p=8192 时，通过公式 2 可计算出：B 的最小需求为 6 GB/s；而在 8×H800 机器上，B 的最小需求会增至 19 GB/s。此外，在实际场景中，由于各传输阶段无法完全重叠，实际所需带宽会更高。但正如我们将在 5.4.2 节中验证的：NVIDIA A800 HGX 服务器中，若每台机器的 100 Gbps 网卡被充分利用，其带宽足以满足上述需求。

3 MOONCAKE 的设计

3.1 整体概述

如图 2 所示，MOONCAKE 采用解耦架构：不仅将预填充实例与解码实例分离，还对 GPU 集群中的 CPU、DRAM、SSD 及 RDMA 资源进行整合，构建分布式 KVCache。为实现对这些解耦组件的调度，MOONCAKE 的核心是一个名为“Conductor”的全局调度器——该调度器会根据当前 KVCache 的分布情况与工作负载特征，实现请求的分发。MOONCAKE Store (详见 3.2 节) 则负责管理这些 KVCache 块的存储与传输。

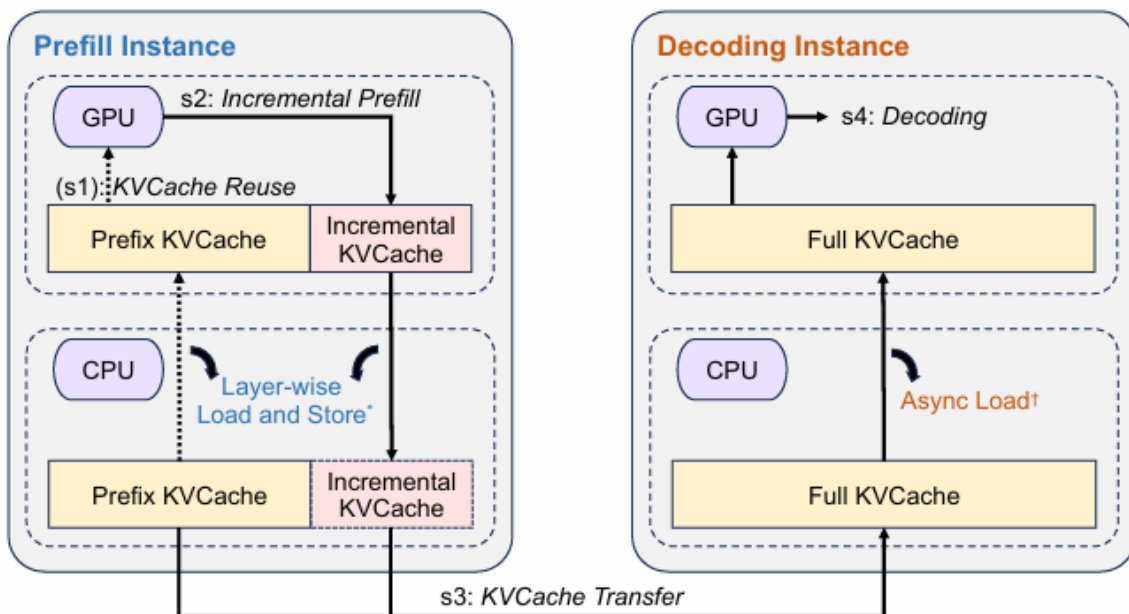


Figure 3: Workflow of inference instances.

具体而言，图 3 展示了单个请求的典型处理流程：在完成 token 化（tokenizing）后，Conductor 会选择一对预填充实例与一个解码实例，并启动包含以下四个步骤的工作流：

- KVCACHE 复用：**被选中的预填充实例（或实例组）会接收请求——该请求包含原始输入、可复用的前缀缓存块密钥（block keys），以及为该请求分配的完整缓存块密钥。实例会根据前缀缓存块密钥，将远程 CPU 内存中的前缀缓存加载到 GPU 内存中，为请求处理“启动预热”；若不存在可复用的前缀缓存，则跳过此步骤。这一步骤的核心目标是平衡三者：最大化 KVCACHE 复用率、均衡不同预填充实例的负载、确保满足 TTFT SLO。这种调度逻辑以 KVCACHE 为核心，后续将在 4 节中进一步阐述。
- 增量预填充：**预填充实例利用前缀缓存完成预填充阶段，并将新生成的“增量 KVCACHE”回存至 CPU 内存。若未缓存的输入 token 数量超过特定阈值，预填充阶段会被拆分为多个“块”（chunk），以流水线方式执行——该阈值的设定需确保充分利用对应 GPU 的计算能力，通常大于 1000 个 token。关于“为何采用分块预填充仍需保留预填充实例与解码实例解耦”的原因，将在 3.3 节中说明。
- KVCACHE 传输：**每个节点中均部署 MOONCAKE Store，负责管理与传输这些缓存数据。此步骤采用异步执行方式，与上述增量预填充步骤并行进行——通过将各模型层生成的 KVCACHE 以流式传输至目标解码实例的 CPU 内存，减少等待时间。
- 解码阶段：**当解码实例的 CPU 内存接收到全部 KVCACHE 后，该请求会以“持续批处理”（continuous batching）的方式加入下一个批处理队列。解码实例由 Conductor 提前选定，选择依据是实例当前的负载状态，以确保不违反 TBT SLO。

3.2 MOONCAKE Store: KVCACHE 的缓存系统

MOONCAKE 的核心是分布式全局 KVCACHE 缓存系统，即 MOONCAKE Store。如 2.2 节所述，复用已缓存的 KVCACHE 不仅能降低计算成本，还能通过缩短 TTFT 提升用户体验——尤其是在聚合带宽被充分利用的场景下。然而，实现带宽的充分利用极具挑战性：其带宽最高可达 8×400 Gbps，与 DRAM 带宽处于同一量级。

本节将分三部分介绍 MOONCAKE Store 的设计：3.2.1 节阐述 KVCACHE 的管理方式，包括存储方案与淘汰策略；3.2.2 节说明 MOONCAKE Store 的对象级 API（应用程序编程接口）与内存传输 API；3.2.3 节详细介绍传输引擎的设计——该引擎是一套高性能、零拷贝的 KVCACHE 传输系统，旨在最大化单台机器上多块 RDMA 网卡的使用收益，通过拓扑感知路径选择、端点池化等技术，提升执行效率与可靠

性。

3.2.1 KVCache 管理

在 MOONCAKE Store 中，所有 KVCache 均以“分页块”（paged block）的形式存储在分布式缓存池中。每个块的大小（即单个块包含的 token 数量）由模型大小与最优网络传输大小共同决定，通常在 16 至 512 个 token 之间。每个块都附带一个哈希密钥（hash key）——该密钥由块自身的哈希值与其前缀的哈希值共同生成，用于实现去重。为缓解热点缓存的访问延迟，同一哈希密钥可在不同节点上存在多个副本，副本数量由 4.2 节所述的“缓存负载均衡策略”控制。

MOONCAKE Store 会在缓存池中为每个缓存块分配存储空间，并记录块密钥、块地址等元数据。当缓存池空间已满时，MOONCAKE Store 会采用“最近最少使用”（LRU, Least Recently Used）策略淘汰现有缓存块——但正在被未完成请求访问的块除外；随后，新块会覆盖被淘汰块的存储空间。

3.2.2 接口设计

在高层级，MOONCAKE Store 提供对象级 API，包括 `put`（存入）、`get`（获取）、`change_replica`（调整副本数量）等。这些 API 支持以解耦方式实现 KVCache 的缓存：将 KVCache 的“迷你块”（mini block）组织为内存对象，并允许 Conductor 通过调整每个 KVCache 块的副本数量，实现更高的带宽聚合。上述功能由一组同步批处理传输 API 提供支持，具体定义如代码清单 1 所示。

代码清单 1：MOONCAKE Store 中的内存传输 API

```
int registerLocalMemory(void *vaddr, size_t len, const string &type);
BatchID allocateBatchID(size_t batch_size);
int submitTransfer(BatchID batch_id, const vector<Request> &entries);
int getTransferStatus(BatchID batch_id, int request_index, Status &status);
int freeBatchID(BatchID batch_id);
```

传输操作同时支持 DRAM 与 GPU VRAM：若指定的内存区域已提前注册，系统会优先采用“GPU Direct RDMA”技术（一种无需 CPU 参与即可实现 GPU 与远程内存直接数据传输的技术）。传输操作的完成状态可通过 `getTransferStatus` API 异步监控——该 API 会返回传输操作的当前状态（正在进行中或已出现错误）。

3.2.3 传输引擎

为高效实现上述 API，MOONCAKE 设计了一套传输引擎，核心目标包括三点：1) 在多块 RDMA 网卡间实现传输任务的高效分配；2) 将 RDMA 连接管理的复杂性对 API 层隐藏；3) 妥善处理临时网络故障。该传输引擎的设计围绕上述目标展开，具体细节如下：

网络配置

MOONCAKE 的性能优势依赖于高带宽的网络互联。目前，系统采用标准 HGX 服务器：每块 A800 GPU 搭配 100/200 Gbps 网卡，每块 H800 GPU 搭配 200/400 Gbps 网卡——这类带宽已与内存带宽处于同一量级，而现有库（除 NCCL 外）均无法充分利用该带宽。此外，NCCL 在节点 / 网卡新增或移除导致的“动态拓扑变化”场景下，无法实现平滑适配，且不支持 DRAM 到 DRAM 的传输路径。与之相比，MOONCAKE 的传输引擎在故障发生时，能主动寻找备选路径。

为缓解网络拥塞，系统采用由云服务商优化后的 RoCEv2（基于融合以太网的远程直接内存访问版本 2）协议；在调度层面，系统通过增加热点 KVCache 的副本数量（详见 4.2 节），进一步减轻拥塞问题。

拓扑感知路径选择

现代推理服务器通常包含多组 CPU 插槽、DRAM、GPU 及 RDMA 网卡设备。尽管从技术上看，本地 DRAM 或 VRAM 的数据可通过任意一块 RDMA 网卡传输至远程节点，但这类传输会受限于 UPI（Ultra Path Interconnect，超路径互联，用于 CPU 插槽间通信）或 PCIe 交换机的带宽瓶颈。为突破这一限制，MOONCAKE Store 实现了“拓扑感知路径选择算法”。

在处理请求前，每个服务器会生成一份“拓扑矩阵”，并在集群内广播该矩阵。该矩阵会针对不同类型的内存（内存类型在注册时指定），将网卡划分为“优先列表”（preferred list）与“备用列表”（secondary list）。正常情况下，系统会从“优先列表”中选择网卡用于传输——例如，仅通过本地 NUMA（非统一内存访问）节点内的 RDMA 操作，或通过本地 PCIe 交换机实现 GPU Direct RDMA；当故障发生时，则可能同时启用“优先列表”与“备用列表”中的网卡。具体流程为：根据内存地址确定对应的本地网卡与目标网卡，建立连接后执行数据传输。

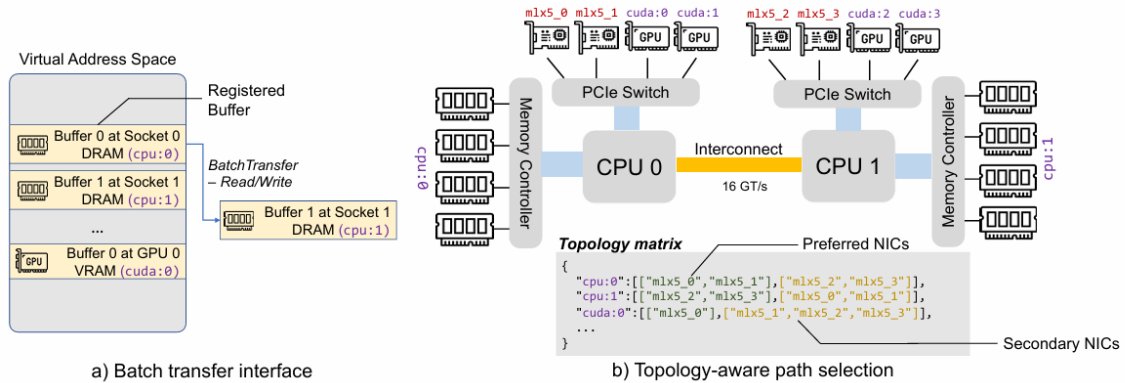


Figure 4: Transfer engine of MOONCAKE Store.

以图 4 所示场景为例：若需将本地节点中“buffer 0”（分配给 cpu:0）的数据传输至目标节点中“buffer 1”（分配给 cpu:1），传输引擎会先通过本地服务器的拓扑矩阵，找到 cpu:0 对应的“优先列表”网卡（如 mlx5_1）作为本地网卡；同理，根据目标内存地址选择目标网卡（如 mlx5_3）。随后，系统会建立从“本地 mlx5_1”到“目标 mlx5_3”的 RDMA 连接，执行 RDMA 读 / 写操作。

为进一步提升带宽利用率，单个请求的传输任务会在内部被划分为多个“切片”（slice），切片大小为 16 KB；每个切片可采用不同的传输路径，从而实现所有 RDMA 网卡的协同工作。

端点管理

MOONCAKE Store 采用“端点对”（pair of endpoints）表示本地 RDMA 网卡与远程 RDMA 网卡之间的连接——实际应用中，每个端点包含一个或多个 RDMA 队列对（queue pair）对象。MOONCAKE Store 中的连接采用“按需建立”的方式：在首个请求到达前，端点均处于未配对状态。

为避免大量端点导致请求处理速度下降，MOONCAKE Store 引入“端点池化”机制，对活跃连接的最大数量进行限制，并采用 SIEVE 算法 [19] 管理端点的淘汰。若某一连接因链路错误中断，该连接会从两端节点的端点池中移除；在下一次数据传输尝试时，系统会重新建立连接。

故障处理

在多网卡环境中，常见的故障场景是“某块网卡临时不可用，但节点间仍存在其他可用传输路径”。

MOONCAKE Store 的设计可高效应对这类临时故障：若检测到某一连接不可用，系统会自动识别可用的备选路径，并将请求重新提交至其他 RDMA 网卡设备。此外，MOONCAKE Store 还能检测 RDMA 上下文、完成队列等其他 RDMA 资源的故障，在故障（如链路中断）解决前，会暂时避免使用这些存在问题的资源。

3.3 MOONCAKE 的预填充池

与功能固定的解码节点不同，“是否需要设计独立且弹性的预填充池”及“预填充池的最佳实现方案”仍是当前研究的争议点。例如，尽管众多研究者 [7-9] 与我们观点一致，认同解耦架构的价值，但随着“分块预填充”技术 [10] 的出现，“预填充与解码是否仍需分离”这一问题值得探讨。

经过深入分析，我们决定在 MOONCAKE 中保留解耦架构，核心原因有两点：首先，在线服务通常对 SLO 有严格要求——分块预填充虽能减少对解码阶段的干扰，但要在“提升预填充阶段 MFU（模型 FLOPs 利用率）”与“满足解码阶段 TBT SLO”之间实现平衡，仍存在显著挑战（这一点将在 5.2 节的端到端实验中验证）。其次，随着近年来 LLM 支持的上下文长度快速增长（从 8k 增至 128k，部分模型甚至可达 100 万 token [20]），预填充节点需要通过不同的跨节点并行策略，应对长上下文请求。对于这类长上下文请求，输入 token 数量通常是输出 token 的 10 至 100 倍，因此“降低 TTFT”成为关键优化目标。由于长上下文预填充具备丰富的并行性，理论上可通过多台 8×GPU 节点并行处理以提升效率；但需注意：若将张量并行（TP）扩展到多台节点，每一层都需执行两次基于 RDMA 的“全归约”（all-reduce）操作——这会导致预填充节点的 MFU 显著下降。

近年来，已有多项研究提出“序列并行”（SP，Sequence Parallelism）技术 [21-27]：通过将请求的输入序列在不同节点间拆分，实现加速，使长上下文请求也能满足 TTFT SLO。然而，序列并行在处理短输入请求时，其 MFU 低于“单节点张量并行”方案；且近期研究 [15] 提出的“弹性序列并行”（通过动态调整序列并行组的规模优化性能）虽具备可行性，但会增加架构复杂度。此外，序列并行仍需频繁的跨节点通信——这不仅会降低 MFU，还会与“节点间 KVCache 传输”争夺网络资源。

为解决上述问题，MOONCAKE 利用“仅解码器 Transformer”的自回归特性，为长上下文预填充设计“分块流水线并行”（CPP，Chunked Pipeline Parallelism）策略：将预填充集群中的每 X 个节点划分为一个“流水线预填充节点组”；对于单个请求，其输入 token 会被拆分为多个“块”（每个块的长度不超过预定义的“预填充块大小”），同一请求的不同块可由不同节点并行处理，从而实现处理加速并降低 TTFT。

分块流水线并行（CPP）具备两大核心优势：1）与训练场景中的流水线并行类似，CPP 仅在“流水线阶段边界”需要跨节点通信——这类通信可轻松与计算过程重叠，因此能实现更高的 MFU，且减少与 KVCache 传输的网络资源竞争；2）CPP 天然适配短上下文与长上下文场景：对短上下文预填充无显著额外开销，且无需通过频繁的节点拆分调整，即可应对长上下文请求。尽管流水线加速技术已在训练系统中得到应用 [28]，但据我们所知，MOONCAKE 是首个将其应用于推理阶段的系统——这一设计主要针对近年来才兴起的“长上下文推理”需求。

4 调度设计

4.1 预填充全局调度

以往关于大语言模型（LLM）部署的研究中，负载均衡策略通常仅依据“已分配请求数量”评估每个实例的负载。而在 MOONCAKE 中，预填充实例的选择需考虑更多因素——不仅包括实例负载，还涉及前缀缓存命中长度及可复用 KVCache 块的分布情况。理论上，将请求路由至“前缀缓存命中长度更长”的预填充实例可减少计算成本，但为确保整个系统的负载均衡并满足首 token 生成时间（TTFT）服务等级目标（SLO），有时需将请求调度至其他节点。为应对这些复杂需求，我们提出一种“感知缓存的全局调度算法”，该算法同时考虑“前缀缓存带来的预填充时间缩短”与“实例本地排队时间”，实现请求的最优分配。

Algorithm 1 KVCache-centric Scheduling Algorithm

Input: prefill instance pool P , decoding instance pool D , request R , cache block size B .

Output: the prefill and decoding instances (p, d) to process R .

```
1:  $block\_keys \leftarrow \text{PrefixHash}(R.prompt\_tokens, B)$ 
2:  $TTFT, p \leftarrow \text{inf}, \emptyset$ 
3:  $best\_len, best\_instance \leftarrow \text{FindBestPrefixMatch}(P, block\_keys)$ 
4: for  $instance \in P$  do
5:   if  $\frac{best\_len}{instance.prefix\_len} > \text{kvcache\_balancing\_threshold}$  then
6:      $prefix\_len \leftarrow best\_len$ 
7:      $transfer\_len \leftarrow best\_len - instance.prefix\_len$ 
8:      $T_{transfer} \leftarrow \text{EstimateKVCacheTransferTime}(transfer\_len)$ 
9:   else
10:     $prefix\_len \leftarrow instance.prefix\_len$ 
11:     $T_{transfer} \leftarrow 0$ 
12:   $T_{queue} \leftarrow \text{EstimatePrefillQueueTime}(instance)$ 
13:   $T_{prefill} \leftarrow \text{EstimatePrefillExecutionTime}(\text{len}(R.prompt\_tokens), prefix\_len)$ 
14:  if  $TTFT > T_{transfer} + T_{queue} + T_{prefill}$  then
15:     $TTFT \leftarrow T_{transfer} + T_{queue} + T_{prefill}$ 
16:     $p \leftarrow instance$ 
17:  $d, TBT \leftarrow \text{SelectDecodingInstance}(D)$ 
18: if  $TTFT > TTFT\_SLO$  or  $TBT > TBT\_SLO$  then
19:   reject  $R$ ; return
20: if  $\frac{best\_len}{p.prefix\_len} > \text{kvcache\_balancing\_threshold}$  then
21:    $\text{TransferKVCache}(best\_instance, p)$ 
22: return  $(p, d)$ 
```

算法 1 详细阐述了这种以 KVCache 为核心的预填充调度机制。对于每个新请求，系统会先将请求的块密钥 (block keys) 与每个预填充实例的缓存密钥逐一比对，确定前缀匹配长度 (prefix_len)。基于这些匹配信息，Conductor 会利用“通过离线数据训练的多项式回归模型”，结合请求长度与各实例的前缀匹配长度 (prefix_len 因实例而异)，估算出请求在不同实例上的执行时间；随后，将该执行时间与实例的预估排队时间相加，得到请求在该实例上的总 TTFT；最终，Conductor 将请求分配给“总 TTFT 最短”的实例，并更新该实例的缓存状态与排队时间。若所有实例均无法满足 TTFT SLO，Conductor 会直接向上层返回 HTTP 429 “请求过多”响应状态码，拒绝该请求。

该调度框架的核心逻辑虽简洁，但各组件的工程实现中隐藏着诸多复杂性。例如，为预测请求在预填充阶段的计算时间，我们采用基于离线测试数据训练的预测模型——该模型会根据请求长度与前缀缓存命中长度估算预填充时长。得益于 Transformer 模型计算模式的规律性，只要拥有充足的离线数据，该预测模型的误差范围就能控制在较小水平。请求的排队时间则通过累加“所有已排队请求的预填充时间”计算得出。在实际实现中，TTFT 的计算采用并行方式处理，因此其耗时相较于推理时间可忽略不计。

调度设计中的更大挑战在于“传输时间预测”：传输时间不仅取决于传输数据量大小，还受当前网络状态影响——尤其是发送节点是否处于拥塞状态。这一问题也凸显了“热点 KVCache 块复制”的必要性，相关策略将在 4.2 节中详细说明。

4.2 缓存负载均衡

在 MOONCAKE 中，每个预填充实例都拥有独立的本地前缀缓存集合，且这些缓存的使用频率差异显著。例如，系统提示词（system prompts）几乎被所有请求访问，而存储“本地长文档内容”的缓存可能仅被单个用户使用。如 4.1 节所述，Conductor 在实现“缓存匹配最大化”与“实例负载均衡”的最优平衡中扮演关键角色；因此，从分布式缓存系统的角度来看，负载均衡同样至关重要——具体而言，需通过合理的缓存备份策略，确保全局预填充调度既能实现高缓存命中率，又能维持低实例负载。

针对 KVCache 调度的一个直观方案可能是：统计每个 KVCache 块的全局使用频率，通过预测模型预估其未来使用情况，再据此制定调度决策。然而，与预填充时间估算不同，LLM 部署的工作负载具有高度动态性，且会随时间发生显著变化——尤其对于用户规模快速增长的模型即服务（MaaS）提供商而言，准确预测 KVCache 块的未来使用情况几乎无法实现。因此，我们提出一种“基于启发式的自动热点迁移方案”，以提升缓存负载均衡效果。

如前所述，由于部分预填充实例负载过高，请求未必总能被调度至“前缀缓存命中长度最长”的实例。在这种情况下，若“额外增加的预填充时间”短于“KVCache 传输时间”，Conductor 会将缓存位置信息与请求一同转发至备选实例；该备选实例会主动从原缓存持有节点获取 KVCache，并将其存储在本地。更重要的是，若“最优远程前缀匹配长度”不超过“当前本地可复用前缀长度与某一阈值的乘积”^①，系统会优先选择重新计算输入 token——而非传输远程 KVCache。这两种策略不仅能减少请求的预填充时间，还能实现热点缓存的自动复制，使热点缓存块在多个实例中分布，进一步提升系统整体效率。

为验证该策略的有效性，我们设计了一组调度对比实验：将“随机调度”“负载均衡调度”与“本文提出的感知缓存负载均衡调度”进行性能比较，同时对比 4.1 节所述的“本地感知缓存调度”与本节所述的“全局感知缓存调度”（考虑缓存负载均衡）。在随机调度中，每个请求会被随机分配给一个预填充实例；在负载均衡调度中，请求会被分配给当前负载最轻的实例。实验基于由 16 个 8×A800 节点组成的 MOONCAKE 集群，采用 5.2.1 节所述的对话负载轨迹进行重放测试，并以 TTFT 为核心评估指标。实验结果如图 5 所示：本文提出的以 KVCache 为核心的调度算法显著优于随机调度与负载均衡调度；其中，引入缓存负载均衡的全局感知缓存算法，相较于仅考虑本地缓存的调度算法，将平均 TTFT 进一步降低了 14%。

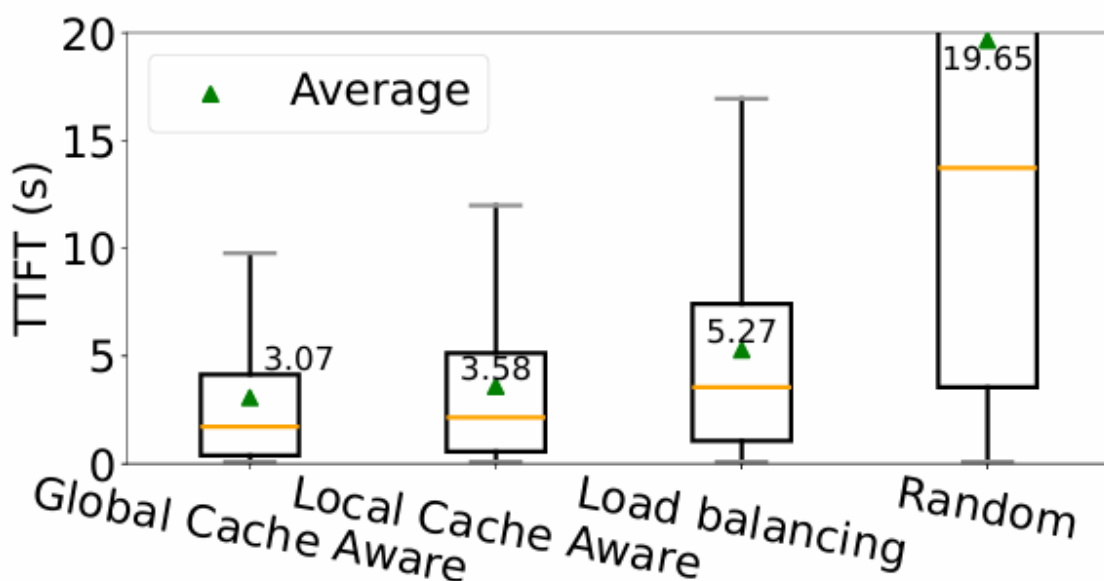


Figure 5: The prefill scheduling experiment.

5 评估

如前文所述，根据 Kimi 的历史统计数据，相较于基于 vLLM 的原有系统，MOONCAKE 使 Kimi 在 A800 和 H800 集群上的请求处理能力分别提升了 115% 和 107%。为进一步验证该结果的有效性并确保可复现性，本节基于一个模拟 LLaMA3-70B 架构的虚拟模型，在 MOONCAKE 上开展了一系列端到端实验与消融实验，旨在解答以下核心问题：1) 在真实场景中，MOONCAKE 是否优于现有 LLM 推理系统？2) 与传统前缀缓存方法相比，MOONCAKE Store 的设计是否能显著提升 MOONCAKE 的性能？

5.1 实验设置

5.1.1 测试环境

在复现实验中，系统部署于一个高性能计算节点集群，以评估其性能表现。集群中每个节点配置 8 块 NVIDIA A800 SXM4-80GB GPU 和 4 块 200 Gbps RDMA 网卡。MOONCAKE Store 中的 KVCache 块大小设为 256。部署 MOONCAKE 时，每个节点根据启动参数，分别作为预填充实例或解码实例运行；部署其他对比系统时，每个节点仅运行一个实例。

5.1.2 评估指标

具体而言，我们测量每个请求的首 token 生成时间 (TTFT) 和 token 间生成时间 (TBT) —— 其中，TBT 取 “token 到达间隔中最长 10%” 的平均值。如 2 节所述，TTFT 阈值设为 30 秒，TBT 阈值根据场景不同分别设为 100 毫秒、200 毫秒和 300 毫秒。我们将 “TTFT 和 TBT 均低于对应阈值” 的请求定义为 “有效请求”，并以 “有效请求在所有请求中的占比” 作为 “有效请求处理能力” 的衡量标准。为简洁起见，后续未提及 TTFT 的实验，均默认满足 TTFT 阈值要求。为更细致地对比缓存性能，我们还测量了预填充阶段的 GPU 计算时间，以及每个请求的缓存命中率。

5.1.3 对比基准

我们采用当前主流的开源 LLM 部署系统 vLLM [14] 作为实验基准。vLLM 具备持续批处理 (continuous batching) 和 PagedAttention 技术，能显著提升推理吞吐量；但其 “预填充与解码阶段耦合” 的架构，在长上下文场景中易对解码过程产生干扰。vLLM 的最新更新已集成 “前缀缓存” 和 “分块预填充” 功能，以改善长上下文场景下的 TTFT 和 TBT 等指标。实验中，我们也对 vLLM 的这些功能进行了对比测试，所使用的 vLLM 版本为最新发布 v0.5.1。受当前实现限制，我们对该版本的 “前缀缓存” 和 “分块预填充” 功能分别进行测试。

5.2 端到端性能

在端到端实验中，我们评估了 MOONCAKE 与基准系统在不同工作负载下的请求处理能力，具体测量 “满足 SLO 阈值的最大吞吐量”。实验采用三类工作负载：两类来自 Kimi 的真实负载轨迹（分别代表在线对话和工具 / 智能体交互场景），以及一类用于覆盖更多推理场景的合成负载。本节将先阐述这些工作负载的独特特征，再分析实验结果，最后通过 “预填充阶段 GPU 计算时间” 的对比，进一步验证 MOONCAKE Store 在提升缓存利用率、降低计算成本方面的优势。

5.2.1 工作负载说明

对话负载

聊天机器人 [1,5] 是 LLM 最广泛的应用场景之一，因此 “对话请求” 是 LLM 推理的典型工作负载。如表 2 所示，该负载包含大量长上下文请求 —— 最长可达 128k token，平均约 12k token，与当前长上下文数据集 [29,30] 的数据长度相当；此外，多轮对话特性使该负载的前缀缓存率平均约为 40%。我们从在线推理集群中采样了 1 小时的对话轨迹，每条轨迹记录包含 “输入 / 输出长度” 和 “请求到达时间戳”。实验中，请求根据时间戳分发，当模型输出达到预设长度时，请求被提前终止。

工具 / 智能体负载

近年来，将 LLM 作为工具或智能体执行任务的研究逐渐增多 [31]。这类任务的典型特征是“包含预定义的长系统提示词”，且这些提示词具有高度重复性。我们同样采集了 1 小时的工具 / 智能体负载轨迹，如表 2 所示，该负载的前缀缓存率更高，但输入和输出长度更短。

合成负载

合成负载基于多个公开数据集构建。我们将真实轨迹中的请求分为三类：短对话、工具 / 智能体调用、长文本摘要与问答，并为每类请求选择对应的数据集：ShareGPT [32]（多轮短对话）、Leval [29]（长上下文基准测试，模拟工具 / 智能体场景中的长系统提示词）、LooGLE [30]（长上下文问答与摘要任务，输入长度最长达 100k token，包含多轮问答和单轮摘要）。整体而言，合成负载的平均输入长度最长；尽管其前缀缓存率最高，但缓存命中分布较为分散，因此对缓存容量的需求更大。

预处理阶段，我们将每轮对话映射为独立请求（包含历史交互的输入与输出）；对于“同一长提示词对应多个问题”的数据集，将“每个问题及其前置提示词”视为单个请求。我们按 1:1:1 的比例组合处理后的数据集，保留多轮对话请求中的序列关系，同时对整体顺序进行随机打乱。由于数据集未指定请求到达时间，实验中通过泊松过程（Poisson process）按设定速率分发请求，模拟真实场景。

表 2：工作负载统计数据

指标	对话负载 (Conversation)	工具 / 智能体负载 (Tool&Agent)	合成负载 (Synthetic)
平均输入长度 (Avg Input Len)	12035	8596	15325
平均输出长度 (Avg Output Len)	343	182	149
缓存率 (Cache Ratio)	40%	59%	66%
到达模式 (Arrival Pattern)	时间戳 (Timestamp)	时间戳 (Timestamp)	泊松过程 (Poisson)
请求数量 (Num Requests)	12031	23608	3993

5.2.2 有效请求处理能力

为评估不同工作负载下“满足 SLO 的最大请求数量”，我们测试了四种系统配置：MOONCAKE、基础版 vLLM、开启前缀缓存的 vLLM、开启分块预填充的 vLLM，每种配置均使用 16 个节点。

对话负载

该负载的实验结果如图 1 所示。由于输入长度可变且输出长度较长，vLLM 系统因“预填充阶段长上下文”的影响，TBT 波动显著。尽管分块预填充能减少对解码的干扰，但“提升预填充阶段 MFU”与“满足解码阶段 TBT 约束”之间的平衡仍难以实现——即使 vLLM 满足 TTFT SLO，其有效请求处理能力仍不理想。相比之下，MOONCAKE 的有效请求处理能力较 vLLM 实现了显著提升。

工具 / 智能体负载

与之不同，工具 / 智能体负载的前缀缓存率高且输出长度短——这一特性对 vLLM 较为有利，因为“短预填充时间”对输出的影响较小。但如图 6 所示，基础版 vLLM 和开启分块预填充的 vLLM，因“预填充处理时间较长”，解码过程受到更严重的干扰，导致其有效缓存能力低于开启前缀缓存的 vLLM。MOONCAKE 通过“全局缓存池”显著提升缓存容量，并通过节点间传输优化缓存利用率，在高前缀缓存场景中表现突出：在 200 毫秒 TBT 阈值下，其有效缓存能力较“开启前缀缓存的 vLLM”提升了 42%。

合成负载

合成负载的平均输入长度最长，且缓存热点分布分散——这导致“小缓存容量”下的缓存利用率极低。如图 7 所示，MOONCAKE 处理的请求中，多数 TBT 可控制在 100 毫秒以内；而 vLLM 处理的请求中，约 20% 的 TBT 超过 300 毫秒。开启前缀缓存或分块预填充的 vLLM，性能与基础版 vLLM 相近，均无法缓解“长上下文对解码阶段的影响”。在 200 毫秒 TBT 阈值下，MOONCAKE 的有效请求处理能力较 vLLM 提升了 40%。

5.2.3 预填充阶段 GPU 时间

预填充阶段的 GPU 时间与请求的 TTFT 及服务成本正相关，且由“请求输入长度”和“缓存命中率”共同决定。我们分析了不同工作负载下预填充阶段的平均 GPU 时间，结果如图 8 所示。对 MOONCAKE 而言，对话负载因“输入长度更长”且“前缀缓存率更低”，预填充 GPU 时间最长；合成负载虽平均输入长度最长，但因“前缀缓存率最高”且“全局缓存池实现了更优缓存命中率”，其预填充 GPU 时间反而短于对话负载；工具 / 智能体负载的平均输入长度最短且前缀缓存率较高，因此预填充 GPU 时间最短。

对比不同系统可见，MOONCAKE 通过“全局缓存充分复用前缀”，显著缩短了 GPU 时间：与 vLLM 相比，在对话、工具 / 智能体、合成负载下，GPU 时间分别减少了 36%、53% 和 64%。开启前缀缓存的 vLLM 依赖 HBM 存储本地缓存，缓存容量远低于 MOONCAKE——其在对话和工具 / 智能体负载下的预填充 GPU 时间，分别是 MOONCAKE 的 1.43 倍和 1.40 倍；而在“缓存热点分散”的合成负载下，开启前缀缓存的 vLLM 与基础版 vLLM 的预填充 GPU 时间接近，且是 MOONCAKE 的 2.59 倍。开启分块预填充的 vLLM，为“维持解码阶段低 TBT”牺牲了部分预填充效率，因此其预填充 GPU 时间最长——在三类负载下，分别是 MOONCAKE 的 1.90 倍、2.68 倍和 3.33 倍。

5.3 MOONCAKE Store 性能评估

为解答“MOONCAKE Store 的全局缓存池对系统性能的影响”这一问题，我们首先通过定量分析明确缓存容量需求，再通过实际工作负载实验验证全局缓存的优势。分析表明：尽管“基于本地 DRAM 构建 KVCache 内存”比“仅使用 HBM”能提升缓存容量，但“单节点本地缓存”仍无法实现最优缓存利用率。

5.3.1 缓存容量需求的定量分析

以 LLaMA3-70B 模型为例，单个 token 所需的 KVCache 大小为 320 KB。尽管每个节点可预留约 1 TB DRAM 用于本地缓存，但仅能存储约 300 万 token，显然无法满足需求。图 9 展示了“不同工作负载及组合负载下”的理论缓存命中率——结果显示，在多数场景中，“300 万 token 容量的本地缓存”无法达到“理论最大命中率的 50%”；而要使缓存命中率接近 100% 的理论最大值，缓存容量需达到 5000 万 token，这需要至少 20 个节点的 DRAM 资源聚合。该结果表明，全局缓存的容量远优于本地缓存，能显著提升缓存命中率、减少 GPU 计算时间。

5.3.2 实际工作负载实验

为评估“全局缓存”与“本地缓存”的效果差异，我们聚焦两个核心指标：缓存命中率和预填充阶段的平均 GPU 计算时间。实验配置 10 个预填充实例节点，将所有请求的输出长度限制为 1（以隔离解码阶段的影响）。本地缓存配置中，每个节点的缓存容量为 300 万 token，但仅能访问自身缓存；全局调度器会将请求路由至“前缀匹配率更高”的节点，以最大化缓存利用率。全局缓存配置中，每个节点的缓存容量同

样为 300 万 token，但支持跨节点共享缓存 —— 通过主动节点间缓存迁移实现。

实验数据如图 10 所示：在所有测试负载下，全局缓存的命中率均更高，且预填充阶段的平均 GPU 计算时间更短。与本地缓存相比，全局缓存的命中率最高提升 136%，预填充计算时间最高减少 48%。

5.3.3 缓存副本分布

基于 4.2 节所述的“缓存负载均衡调度策略”，MOONCAKE Store 中的缓存密钥（cache key）可在不同机器上存储副本，以降低热点缓存的访问延迟。为进一步分析系统动态特性，我们统计了三类负载下“缓存密钥的副本数量”，结果如图 11 所示。

可见，在对话和工具 / 智能体负载中，存在高度集中的热点缓存（例如前 100 个密钥）—— 系统稳定后，这些热点缓存的副本几乎覆盖预填充池中的所有实例；而合成负载的共享前缀缓存较少，因此即使是前 10 个缓存块，其副本数量也较少且存在波动。这一结果验证了 4.2 节所述调度策略的有效性：能为热点缓存（尤其在“前缀缓存高度集中”的场景中）高效提供副本。

5.4 KVCache 传输性能

5.4.1 传输引擎性能

MOONCAKE 的传输引擎旨在实现节点间高效的缓存传输。我们将其延迟与其他主流方案对比，选择两类对比基准：基于 Gloo 后端的 torch.distributed，以及基于 TCP 的传输方案。所有方案的并发度均设为 64，最小传输粒度设为 128 KB。

如图 12 所示，MOONCAKE 的传输引擎延迟显著低于对比方案。传输 40 GB 数据（对应 LLaMA3-70B 模型 128k token 的缓存大小）时，在“4×200 Gbps”和“8×400 Gbps”的网络配置下，传输引擎的带宽分别达到 87 GB/s 和 190 GB/s，约为 TCP 协议的 2.4 倍和 4.6 倍。该传输引擎的代码后续也将开源 —— 因其具备解耦特性，可作为基础工具应用于多种场景（例如，目前已用于 Moonshot AI 的检查点传输服务）。

5.4.2 MOONCAKE 的带宽需求

MOONCAKE 的全局缓存池依赖“高效节点间缓存传输”，以“将缓存传输时间隐藏在 GPU 计算时间内”。为评估网络带宽对系统性能的影响，我们模拟了 24 Gbps 至 400 Gbps 的带宽范围，在 5.2.1 节所述的合成负载下，测量传输时间和 TTFT。

图 13a 显示，随着带宽增加，请求的平均 TTFT 逐渐降低：当总通信带宽超过 100 Gbps 时，平均 TTFT 可稳定在 2 秒以内，显著低于“重新计算”的 TTFT 基准；而当带宽低于 100 Gbps 时，系统性能大幅下降 —— TTFT 急剧增加，且“实际传输时间与理论传输时间”出现显著偏差（如图 13b 所示），表明网络已发生严重拥塞。因此，我们建议将 100 Gbps 作为“确保系统最优性能”的最小网络带宽。

5.4.3 端到端延迟分解

MOONCAKE 中单个推理请求的延迟可分解为五个部分：1) 调度与排队时间；2) 分层预填充时间；3) 缓存传输时间；4) 解码节点“将缓存从 DRAM 加载到 HBM”的时间；5) 解码时间。我们在“前缀缓存率 0%”和“前缀缓存率 95%”两种设置下，实验分析了各部分延迟的占比，结果如图 14 所示。

首先，前缀缓存的引入显著缩短了预填充时间：当输入长度为 128k token 时，前缀缓存使预填充时间减少 92%。其次，MOONCAKE 引入的额外开销对系统性能影响极小：调度、传输、缓存加载这三个环节可与模型推理异步执行，因此不影响吞吐量；且“这些开销导致的 TTFT 增加”，远小于“前缀缓存带来的 TTFT 减少”—— 即使计入额外开销，在输入长度为 128k token 的场景中，MOONCAKE 的前缀缓存仍能使 TTFT 减少 86%。

5.5 预填充 / 解码节点比例 (P/D Ratio)

作为已部署的“预填充 / 解码解耦系统”，本节探讨“不同预填充节点与解码节点比例 (P/D Ratio)”对系统性能的影响。实验基于 16 个节点的集群，通过调整 P/D 比例，在 5.2.1 节所述的合成负载下测量平均 TTFT 和 TBT，再结合 5.2.2 节定义的“有效请求处理能力” (TTFT 阈值 10 秒、TBT 阈值 100 毫秒) 进行分析。

结果显示，增加预填充节点数量会缩短 TTFT，但会增加 TBT；反之亦然 (如图 15b 所示)。因此，需在 TTFT 和 TBT 之间寻找平衡。图 15a 表明，当 P/D 比例约为 1:1 时，MOONCAKE 的有效请求处理能力达到最高——这意味着此时预填充集群与解码集群的负载相对均衡。

我们还注意到，已有研究 [7,9] 提出“动态切换节点的预填充 / 解码角色”。但在实际部署中，我们发现在线流量的统计特征通常较为稳定，因此选择“固定 P/D 比例”，同时持续监控预填充与解码集群的负载——仅当负载出现显著波动时，才切换节点角色。

6 相关工作

目前，学术界与工业界已围绕**调度优化**、**内存管理**、**资源解耦**三大方向，开展了大量旨在提升 LLM 部署系统效率的研究工作。其中，FasterTransformer [33]、TensorRT-LLM [34]、DeepSpeed Inference [35] 等生产级系统，通过底层优化显著提升了推理吞吐量；Orca [13] 采用迭代级调度策略，实现了不同阶段请求的并发处理；vLLM [14] 则借助动态 KVCache 管理技术优化内存使用效率。此外，FlexGen [36]、Sarathi-Serve [10]、FastServe [37] 等系统通过创新的调度与缓存交换策略，在有限硬件资源下实现了工作负载的高效分配，这些方案的优化方向各有侧重，且常可形成互补。进一步地，已有研究 [7-9] 通过分离预填充与解码阶段实现了架构解耦，为 MOONCAKE 的设计提供了重要参考。需要说明的是，MOONCAKE 的设计借鉴了开源社区 (尤其是 vLLM) 的技术成果，在此表示衷心感谢。

前缀缓存作为减少 LLM 推理计算开销的关键技术，也被广泛应用于各类推理系统 [14,34]。例如，Prompt Cache [16] 通过在推理服务器上预计算并存储高频文本的 KVCache，实现缓存复用并显著降低推理延迟；SGLang [17] 则提出 RadixAttention 技术，在基数树 (radix tree) 结构中集成最近最少使用 (LRU) 缓存，可自动适配多种复用场景，高效实现跨请求的 KVCache 共享。

在相关研究中，与本文工作同期开展的 CachedAttention [38] 提出了一种分层 KV 缓存系统——该系统利用低成本存储介质 (如 DRAM、SSD) 存储所有请求的 KVCache，与 MOONCAKE 的架构设计存在诸多共通点。但需注意的是，在长上下文推理场景中，KVCache 体积会急剧增大，不仅对缓存容量提出更高要求，还需要高效的数据传输机制与“以 KVCache 为核心的全局调度策略”支撑；此外，MOONCAKE 并非独立的缓存服务，而是融合了“内存高效的缓存存储机制”与“感知缓存的调度策略”，进一步提升了前缀缓存的复用效率。

值得一提的是，分布式 KVCache 池的收益依赖于缓存命中率——在缓存容量固定的前提下，“单位 token 缓存大小”越小，命中率越高。因此，KVCache 压缩 [39-41]、KVCache 友好型注意力架构 [42,43] 等正交技术，可与本文提出的 MOONCAKE 架构形成协同，进一步提升系统性能。

7 Conclusion

This paper presents MOONCAKE, a KVCache-centric disaggregated architecture designed for efficiently serving LLMs, particularly in long-context scenarios. We discuss the necessity, challenges, and design choices involved in balancing the goal of maximizing overall effective throughput while meeting latency-related SLO requirements.